UNIFIED DEVELOPER IMPLEMENTATION BLUEPRINT v4.0

Executive OS + Behavioral Intelligence OS

Production Build Specification — Single Source of Truth

Architecture: 3-Plane Model + 4-Layer Behavioral Intelligence + Multi-Modal Pipeline

Target Platform: WhatsApp Business API + Apple Messages for Business + Voice

LLM Provider: Multi-Provider (OpenAI + Anthropic + Google + Local) via Unified Router

Database: PostgreSQL 16 + pgvector + Redis 7 (19 tables) + Knowledge Graph

Primary Language: Python 3.12+ (FastAPI + Temporal)

Cost Target: < $0.035 per interaction blended

Knowledge Files: 9 structured .md files per user, versioned

Security: Privilege-Isolated Prompt Injection Defense + Capability-Based Access

February 2026 • CONFIDENTIAL

Table of Contents

1. Unified System Architecture — 3-Plane + 4-Layer + Multi-Modal

The system decomposes into **three independently deployable planes** (Gateway, Brain, Hands) with a **4-layer Behavioral Intelligence system** (Brain Layer, DNA, Bones, Muscles) embedded within the Brain Plane, and a **Multi-Modal Pipeline** in the Gateway Plane. Each plane has a clear responsibility boundary, its own scaling policy, and can fail without taking down the others.

1.1 Architecture Principle

Intelligence can fail safely while the system stays deterministic. The Gateway never crashes because the Brain hallucinated. The Brain never stalls because a Google API timed out in the Hands. Plane isolation is enforced by network policies, separate process groups, and independent health checks. The 4-layer Behavioral Intelligence system lives entirely within the Brain Plane, augmenting every intelligence decision with deep user context. **New in v4.0:** The Multi-Modal Pipeline handles voice, image, and document processing at the Gateway level before routing to the Brain, and the Multi-Provider LLM Router in the Brain Plane eliminates single-provider dependency.

1.2 Plane Definitions

Plane	Responsibility	Key Services	Failure Mode	v4.0 Additions
Gateway	Ingress, auth, session management, rate limiting, channel adaptation, message routing, webhook handling, multi-modal preprocessing	WhatsApp Webhook Receiver, iMessage Relay, Auth Service, Session Store (Redis), Rate Limiter, Channel Adapter, Voice Transcription Pipeline, Image Processing Pipeline, Document Ingestion Pipeline	Returns 503 with queued retry. User sees 'processing' indicator.	Voice-to-text (Whisper), Image preprocessing, Document parsing, Streaming progress reporter
Brain	All intelligence: intent classification, tier routing, planning, LLM orchestration, memory retrieval, context compilation, response generation, guardrails + 4-Layer Behavioral Intelligence + emotional intelligence	Tier Router, Intent Classifier, Planner (T0-T3), Multi-Provider LLM Router, Context Compiler, Memory Engine, Knowledge Compiler, Context Injector, Behavioral Rule Engine, Feedback Processor, Profiling Engine, Emotion Classifier, Delegation Tracker, Research Orchestrator	Degrades to cached responses + read-only mode. Knowledge files serve from Redis cache.	Multi-provider LLM router, Hierarchical task decomposition, Emotion classifier, Team awareness engine, Research orchestrator
Hands	All external actions: tool execution, OAuth token management, delta-sync, side-effect commits, webhook dispatch, background jobs, proactive triggers, nightly consolidation, weekly self-review, document generation, MCP client	Tool Executor, OAuth Vault, Webhook Handler, Delta-Sync Workers, Job Scheduler (BullMQ), Side-Effect Ledger, Proactive Trigger Engine, Consolidation Worker, Self-Review Worker, MCP Client Hub, Document Generator, Voice TTS Engine	Read operations continue via cache. Write operations queued. User told 'actions paused.'	MCP client hub, Document generator (PDF/DOCX), Voice TTS output, Delegation messenger, Workflow executor

1.3 The 4-Layer Behavioral Intelligence System (Inside Brain Plane)

The Behavioral Intelligence system transforms a generic assistant into a deeply personalized digital chief of staff. It generates and maintains structured knowledge files per user through conversational extraction, behavioral inference, and continuous learning.

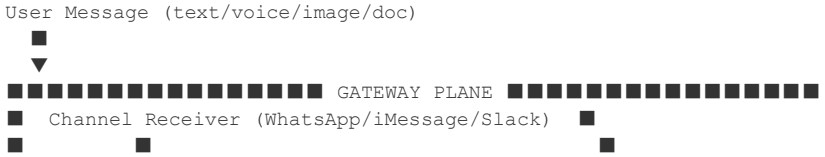
Layer	Purpose	Input Method	Output Files	Update Freq.	v4.0 Changes
BRAIN (L1)	Deep user profiling. Who the user is, how they think, what they want, what frustrates them.	Conversational interview (onboarding + progressive) + behavioral inference from usage	USER.md, SOUL.md, IDENTITY.md, HEARTBEAT.md, TEAM.md	Real-time writes, weekly deep review	Added TEAM.md generation, emotional state tracking
DNA (L2)	Behavioral operating logic. How the agent should think, decide, escalate, handle uncertainty, learn from mistakes.	Derived from Brain profiling + explicit user preferences + agent self-learning	AGENTS.md, MEMORY.md, WORKFLOWS.md	After each correction, nightly consolidation	Added WORKFLOWS.md, delegation rules, research preferences
BONES (L3)	Structural knowledge. Codebase maps, tool ecosystems, conventions, stability maps, tribal knowledge.	Repository scanning + user interview about tech stack + code analysis	skills/[repo]/SKILL.md, TOOLS.md, MEMORY.md, HEARTBEAT.md	On code change, weekly repo scan	MCP server catalog, capability mapping
MUSCLES (L4)	Model and tool orchestration. Which AI model for which task, cost routing, MCP connections, fallback chains.	User interview about AI tools + usage analytics + cost monitoring + provider health	TOOLS.md, AGENTS.md, MEMORY.md, HEARTBEAT.md	On model change, daily cost tracking, real-time failover	Multi-provider routing table, failover chains, cost optimization, MCP registry

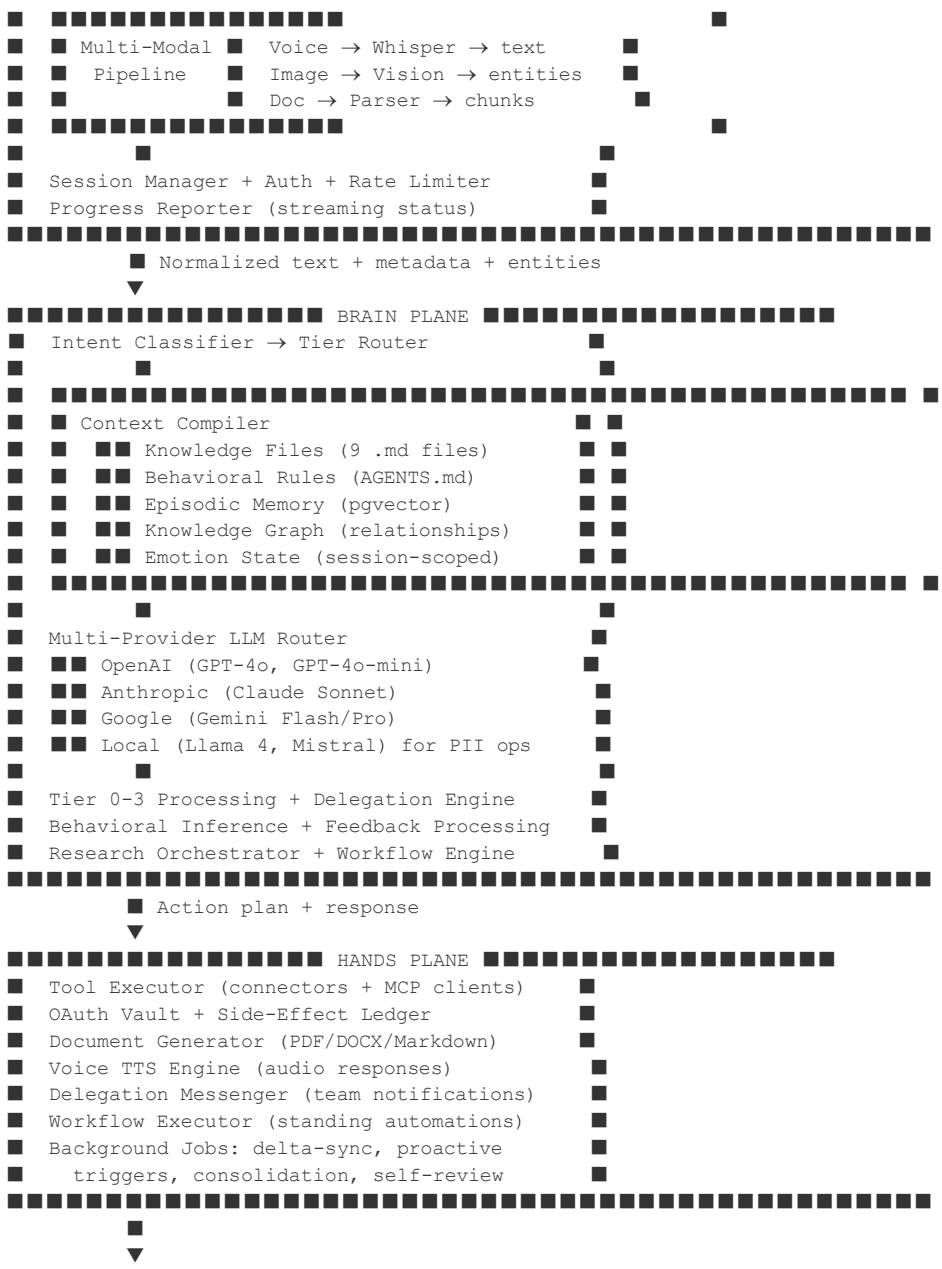
1.4 Multi-Modal Pipeline (Inside Gateway Plane) [NEW]

All non-text input is normalized to structured text + metadata before reaching the Brain Plane. This ensures the Brain's intelligence engine remains modality-agnostic while supporting rich input.

Modality	Input Source	Processing Pipeline	Output to Brain	Cost per Unit
Voice	WhatsApp voice notes, iMessage audio	Whisper API (large-v3) → text + language detection + speaker diarization	Transcribed text + confidence score + duration metadata	$0.006/min
Image	Photos, screenshots, business cards, receipts, whiteboard photos	GPT-4o Vision → structured extraction (contacts, text, action items, data)	Extracted entities + classification (business_card, receipt, whiteboard, screenshot, document)	$0.003-0.01/image
Document	PDF, DOCX, XLSX attachments forwarded via messaging	Document parser (unstructured.io) → chunked text + table extraction	Document summary + key entities + action items + full text chunks for RAG	$0.002-0.008/doc
Voice Out	Agent response flagged for audio delivery	OpenAI TTS (tts-1) or ElevenLabs → audio file	Audio attachment sent via channel adapter	$0.015/1K chars

1.5 System Data Flow (Updated)




Multi-Modal Pipeline
Voice → Whisper → text
Image → Vision → entities
Doc → Parser → chunks
Session Manager + Auth + Rate Limiter
Progress Reporter (streaming status)
Normalized text + metadata + entities
BRAIN PLANE
Intent Classifier → Tier Router
Context Compiler
Knowledge Files (9 .md files)
Behavioral Rules (AGENTS.md)
Episodic Memory (pgvector)
Knowledge Graph (relationships)
Emotion State (session-scoped)
Multi-Provider LLM Router
OpenAI (GPT-4o, GPT-4o-mini)
Anthropic (Claude Sonnet)
Google (Gemini Flash/Pro)
Local (Llama 4, Mistral) for PII ops
Tier 0-3 Processing + Delegation Engine
Behavioral Inference + Feedback Processing
Research Orchestrator + Workflow Engine
Action plan + response
HANDS PLANE
Tool Executor (connectors + MCP clients)
OAuth Vault + Side-Effect Ledger
Document Generator (PDF/DOCX/Markdown)
Voice TTS Engine (audio responses)
Delegation Messenger (team notifications)
Workflow Executor (standing automations)
Background Jobs: delta-sync, proactive
triggers, consolidation, self-review
Response (text/audio/document/rich card)
→ Channel Adapter → User

2. Tech Stack & Infrastructure

Category	Technology	Notes
Runtime	Python 3.12+	FastAPI for HTTP, Temporal for durable workflows, async throughout
Database	PostgreSQL 16 + pgvector	19 tables, RLS on all, HNSW index for embeddings, JSONB for knowledge graph edges
Cache	Redis 7	Session store, knowledge file hot cache, semantic cache, rate limiting, pub/sub for progress streaming
Queue	BullMQ (Redis-backed)	Delta-sync, proactive triggers, consolidation, self-review, workflow execution
Orchestration	Temporal	T3 multi-step plans, saga compensation, research workflows, long-running delegations
LLM — Primary	OpenAI GPT-4o + GPT-4o-mini	Main inference (T1-T3), classification, extraction, knowledge compilation
LLM — Secondary	Anthropic Claude Sonnet 4	Email drafting, nuanced communication, safety validation, injection defense
LLM — Tertiary	Google Gemini Flash/Pro	Long context tasks, failover, cost optimization for bulk processing
LLM — Local	Llama 4 / Mistral (vLLM)	PII-heavy operations, air-gapped processing, offline fallback
LLM — Router	Custom (LiteLLM-inspired)	Unified interface, automatic failover, task-optimized routing, cost tracking
Voice — STT	OpenAI Whisper (large-v3)	Voice note transcription, language detection, ~$0.006/min
Voice — TTS	OpenAI TTS-1 / ElevenLabs	Audio responses for briefings, hands-free mode
Vision	GPT-4o Vision	Business card scanning, receipt capture, whiteboard extraction, screenshot analysis
Document Parse	Unstructured.io	PDF/DOCX/XLSX ingestion, table extraction, chunking for RAG
Embeddings	text-embedding-3-small	1536 dims, HNSW index (ef_construction=200, m=16), monthly re-embed check
Web Search	Tavily API	Structured search results, research monitoring, competitive intelligence
MCP Client	Custom Python MCP SDK	Model Context Protocol for third-party tool integration
Auth	Clerk	User authentication, multi-channel identity linking
OAuth	Custom vault + AWS SM	AES-256-GCM encryption, auto-refresh, scoped access per connector
Infra	AWS ECS Fargate	Per-plane scaling, VPC isolation, ALB for Gateway
Storage	S3	Attachments, knowledge snapshots, document generation output, voice files
Monitoring	Axiom + OpenTelemetry	Distributed tracing, custom metrics, cost tracking per LLM call
CI/CD	GitHub Actions	Per-plane deploys, contract tests, agentic eval suite, prompt injection fuzzing
Secrets	AWS Secrets Manager	Rotated, scoped per plane, audited access

3. Merged Database Schema — 19 Tables (Complete SQL)

All SQL is production-ready. Copy-paste into migration files. Row-Level Security (RLS) is enforced on every table. All timestamps are timestamptz. UUIDs use gen_random_uuid(). The schema merges 10 core Executive OS tables with 5 Behavioral Intelligence tables and **4 new tables** for team management, workflows, knowledge graph, and research jobs.

3.1 Extensions & Enums

```
-- Required extensions
CREATE EXTENSION IF NOT EXISTS "pgcrypto";      -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS "vector";         -- pgvector for embeddings
CREATE EXTENSION IF NOT EXISTS "pg_trgm";        -- fuzzy text matching

-- Core Enums
CREATE TYPE channel_type AS ENUM ('whatsapp', 'imessage', 'slack', 'web', 'voice');
CREATE TYPE message_direction AS ENUM ('inbound', 'outbound');
CREATE TYPE input_modality AS ENUM ('text', 'voice', 'image', 'document', 'location');
CREATE TYPE run_state AS ENUM (
  'pending', 'classifying', 'planning', 'executing',
  'awaiting_approval', 'completed', 'failed', 'cancelled',
  'delegated', 'researching'
);
CREATE TYPE memory_type AS ENUM ('preference', 'fact', 'contact', 'procedure', 'episode');
CREATE TYPE sensitivity_level AS ENUM ('public', 'private', 'sensitive');
CREATE TYPE approval_status AS ENUM ('pending', 'approved', 'denied', 'expired');
CREATE TYPE tool_exec_status AS ENUM ('success', 'failed', 'timeout', 'compensated');
CREATE TYPE risk_level AS ENUM ('none', 'low', 'medium', 'high', 'critical');
CREATE TYPE account_status AS ENUM ('active', 'expired', 'revoked');
CREATE TYPE trigger_type AS ENUM ('schedule', 'event', 'pattern', 'goal', 'research', 'workflow', 'delegation');
CREATE TYPE emotion_state AS ENUM ('neutral', 'positive', 'frustrated', 'rushed', 'stressed', 'excited');
CREATE TYPE llm_provider AS ENUM ('openai', 'anthropic', 'google', 'local', 'mcp');

-- Behavioral Intelligence Enums
CREATE TYPE bi_layer AS ENUM ('brain', 'dna', 'bones', 'muscles');
CREATE TYPE profiling_status AS ENUM ('pending', 'active', 'completed', 'skipped');
CREATE TYPE feedback_type AS ENUM (
  'correction', 'override', 'edit', 'praise', 'complaint', 'restyle'
);
CREATE TYPE rule_source AS ENUM ('explicit', 'inferred', 'correction', 'default');
CREATE TYPE goal_status AS ENUM ('active', 'completed', 'paused', 'abandoned');
CREATE TYPE goal_timeframe AS ENUM ('this_week', 'this_month', 'this_quarter', 'this_year', '3_year');

-- v4.0 New Enums
CREATE TYPE delegation_status AS ENUM ('pending', 'sent', 'acknowledged', 'in_progress', 'completed', 'overdue', 'cancel
CREATE TYPE research_status AS ENUM ('active', 'paused', 'completed', 'archived');
CREATE TYPE workflow_status AS ENUM ('active', 'paused', 'draft', 'archived');
CREATE TYPE workflow_trigger_type AS ENUM ('schedule', 'event', 'condition', 'manual');
CREATE TYPE content_provenance AS ENUM ('user_direct', 'email_body', 'web_search', 'calendar_desc', 'document', 'mcp_res
```

3.2 Core Tables (10 tables — unchanged IDs, enhanced columns)

```
-- 1. accounts
CREATE TABLE accounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  clerk_user_id TEXT UNIQUE NOT NULL,
  display_name TEXT NOT NULL,
  timezone TEXT NOT NULL DEFAULT 'America/New_York',
  status account_status NOT NULL DEFAULT 'active',
  preferred_channel channel_type DEFAULT 'whatsapp',
```

```
  quiet_hours_start TIME,
  quiet_hours_end TIME,
  max_daily_proactive INT DEFAULT 5,
  onboarding_completed_at TIMESTAMPTZ,
  emotion_sensitivity FLOAT DEFAULT 0.5,  -- NEW: how much to adapt to emotional state
  team_enabled BOOLEAN DEFAULT false,      -- NEW: team features toggle
  voice_enabled BOOLEAN DEFAULT false,     -- NEW: voice I/O toggle
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 2. channel_connections
CREATE TABLE channel_connections (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  channel channel_type NOT NULL,
  channel_identifier TEXT NOT NULL,        -- phone number, apple ID, slack user
  is_primary BOOLEAN DEFAULT false,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(channel, channel_identifier)
);

-- 3. messages
CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  run_id UUID REFERENCES runs(id),
  channel channel_type NOT NULL,
  direction message_direction NOT NULL,
  content TEXT NOT NULL,
  input_modality input_modality DEFAULT 'text',     -- NEW
  original_media_url TEXT,                          -- NEW: S3 URL for voice/image/doc
  transcription_confidence FLOAT,                   -- NEW: Whisper confidence
  extracted_entities JSONB DEFAULT '{}',            -- NEW: from multi-modal pipeline
  content_provenance content_provenance DEFAULT 'user_direct',  -- NEW: for security
  emotion_detected emotion_state DEFAULT 'neutral',   -- NEW
  wa_message_id TEXT,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_messages_user_created ON messages(user_id, created_at DESC);
CREATE INDEX idx_messages_run ON messages(run_id);

-- 4. runs (enhanced with delegation + research tracking)
CREATE TABLE runs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  trigger_message_id UUID REFERENCES messages(id),
  intent TEXT NOT NULL,
  tier INT NOT NULL DEFAULT 1 CHECK (tier BETWEEN 0 AND 3),
  state run_state NOT NULL DEFAULT 'pending',
  llm_provider llm_provider DEFAULT 'openai',         -- NEW: which provider handled this
  context_tokens_used INT DEFAULT 0,
  knowledge_files_injected TEXT[] DEFAULT '{}',        -- NEW: which files were used
  plan JSONB,
  result JSONB,
  error JSONB,
  parent_run_id UUID REFERENCES runs(id),
  delegation_id UUID,                                   -- NEW: links to delegations table
  research_job_id UUID,                                 -- NEW: links to research_jobs table
  cost_cents FLOAT DEFAULT 0,
  latency_ms INT,
  created_at TIMESTAMPTZ DEFAULT now(),
  completed_at TIMESTAMPTZ
);
CREATE INDEX idx_runs_user_state ON runs(user_id, state);
```

```
-- 5. memories (unchanged)
CREATE TABLE memories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  run_id UUID REFERENCES runs(id),
  memory_type memory_type NOT NULL,
  content TEXT NOT NULL,
  embedding vector(1536),
  sensitivity sensitivity_level DEFAULT 'private',
  confidence FLOAT DEFAULT 0.8 CHECK (confidence BETWEEN 0 AND 1),
  source TEXT,
  expires_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_memories_embedding ON memories USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 200);
CREATE INDEX idx_memories_user_type ON memories(user_id, memory_type);

-- 6. tool_executions (enhanced with MCP + provenance)
CREATE TABLE tool_executions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_id UUID NOT NULL REFERENCES runs(id),
  user_id UUID NOT NULL REFERENCES accounts(id),
  tool_name TEXT NOT NULL,
  is_mcp BOOLEAN DEFAULT false,                   -- NEW: was this via MCP
  mcp_server_id TEXT,                              -- NEW: which MCP server
  arguments JSONB NOT NULL,
  input_provenance content_provenance DEFAULT 'user_direct',  -- NEW
  result JSONB,
  status tool_exec_status NOT NULL DEFAULT 'success',
  risk_level risk_level DEFAULT 'none',
  approval_status approval_status,
  idempotency_key TEXT UNIQUE,
  compensating_action JSONB,                       -- NEW: saga compensation data
  cost_cents FLOAT DEFAULT 0,
  latency_ms INT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_tool_exec_run ON tool_executions(run_id);

-- 7. oauth_tokens (unchanged)
CREATE TABLE oauth_tokens (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  provider TEXT NOT NULL,
  encrypted_access_token TEXT NOT NULL,
  encrypted_refresh_token TEXT,
  token_expiry TIMESTAMPTZ,
  scopes TEXT[] DEFAULT '{}',
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(user_id, provider)
);

-- 8. proactive_triggers (enhanced with research + workflow + delegation types)
CREATE TABLE proactive_triggers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  trigger_type trigger_type NOT NULL,
  source TEXT NOT NULL,
  payload JSONB NOT NULL,
  confidence_threshold FLOAT DEFAULT 0.7,
  fire_at TIMESTAMPTZ,
  fired_at TIMESTAMPTZ,
  dismissed BOOLEAN DEFAULT false,
  fire_count INT DEFAULT 0,
  user_dismissed_count INT DEFAULT 0,
```

```
  workflow_id UUID,                              -- NEW: links to workflows table
  delegation_id UUID,                            -- NEW: links to delegations table
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_triggers_user_fire ON proactive_triggers(user_id, fire_at);

-- 9. sync_cursors (unchanged)
CREATE TABLE sync_cursors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  provider TEXT NOT NULL,
  resource_type TEXT NOT NULL,
  cursor_value TEXT NOT NULL,
  last_sync_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(user_id, provider, resource_type)
);

-- 10. side_effects (unchanged)
CREATE TABLE side_effects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_id UUID NOT NULL REFERENCES runs(id),
  user_id UUID NOT NULL REFERENCES accounts(id),
  effect_type TEXT NOT NULL,
  description TEXT NOT NULL,
  reversible BOOLEAN DEFAULT false,
  reversed_at TIMESTAMPTZ,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now()
);
```

3.3 Behavioral Intelligence Tables (5 tables — enhanced)

```
-- 11. knowledge_files
CREATE TABLE knowledge_files (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  file_path TEXT NOT NULL,                   -- e.g., 'USER.md', 'TEAM.md', 'WORKFLOWS.md'
  layer bi_layer NOT NULL,
  content TEXT NOT NULL,
  content_hash TEXT NOT NULL,
  token_count INT NOT NULL,
  version INT NOT NULL DEFAULT 1,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(user_id, file_path, version)
);
CREATE INDEX idx_kf_user_path ON knowledge_files(user_id, file_path, version DESC);

-- 12. profiling_sessions
CREATE TABLE profiling_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  dimension TEXT NOT NULL,
  layer bi_layer NOT NULL DEFAULT 'brain',
  status profiling_status NOT NULL DEFAULT 'pending',
  questions_asked INT DEFAULT 0,
  facts_extracted INT DEFAULT 0,
  progress_pct FLOAT DEFAULT 0,
  scheduled_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 13. behavioral_rules
CREATE TABLE behavioral_rules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
```

```
  file_path TEXT NOT NULL,
  category TEXT NOT NULL,
  rule_key TEXT NOT NULL,
  rule_value TEXT NOT NULL,
  source rule_source NOT NULL DEFAULT 'default',
  confidence FLOAT DEFAULT 0.5 CHECK (confidence BETWEEN 0 AND 1),
  override_count INT DEFAULT 0,
  last_triggered_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(user_id, file_path, category, rule_key)
);

-- 14. feedback_signals
CREATE TABLE feedback_signals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  run_id UUID REFERENCES runs(id),
  signal_type feedback_type NOT NULL,
  original_output TEXT,
  corrected_output TEXT,
  context JSONB DEFAULT '{}',
  lesson_extracted JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_feedback_user ON feedback_signals(user_id, created_at DESC);

-- 15. goals
CREATE TABLE goals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  title TEXT NOT NULL,
  description TEXT,
  timeframe goal_timeframe NOT NULL,
  status goal_status NOT NULL DEFAULT 'active',
  progress_pct FLOAT DEFAULT 0,
  milestones JSONB DEFAULT '[]',
  blockers JSONB DEFAULT '[]',
  target_date DATE,
  last_activity_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
```

3.4 New v4.0 Tables (4 tables)

```
-- 16. delegations [NEW]
CREATE TABLE delegations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  delegate_name TEXT NOT NULL,               -- who is being delegated to
  delegate_contact TEXT,                      -- email, phone, slack handle
  delegate_channel channel_type,
  task_description TEXT NOT NULL,
  context JSONB DEFAULT '{}',                -- relevant background info
  status delegation_status NOT NULL DEFAULT 'pending',
  priority risk_level DEFAULT 'medium',
  deadline TIMESTAMPTZ,
  reminder_schedule JSONB DEFAULT '[]',      -- array of reminder timestamps
  last_reminder_at TIMESTAMPTZ,
  completion_criteria TEXT,
  result TEXT,
  linked_run_id UUID REFERENCES runs(id),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_delegations_user_status ON delegations(user_id, status);
```

```
-- 17. research_jobs [NEW]
CREATE TABLE research_jobs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  title TEXT NOT NULL,
  query TEXT NOT NULL,
  sources TEXT[] DEFAULT '{}',              -- specific sources to monitor
  schedule TEXT,                             -- cron expression or 'once'
  status research_status NOT NULL DEFAULT 'active',
  last_run_at TIMESTAMPTZ,
  next_run_at TIMESTAMPTZ,
  findings JSONB DEFAULT '[]',              -- accumulated research findings
  delivery_channel channel_type,
  delivery_format TEXT DEFAULT 'summary',    -- summary, detailed, bullet_points
  max_cost_per_run FLOAT DEFAULT 0.50,
  total_cost FLOAT DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 18. workflows [NEW]
CREATE TABLE workflows (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  name TEXT NOT NULL,
  description TEXT,
  trigger_type workflow_trigger_type NOT NULL,
  trigger_config JSONB NOT NULL,            -- cron, event match, condition
  actions JSONB NOT NULL,                   -- ordered list of action steps
  status workflow_status NOT NULL DEFAULT 'active',
  last_run_at TIMESTAMPTZ,
  next_run_at TIMESTAMPTZ,
  run_count INT DEFAULT 0,
  error_count INT DEFAULT 0,
  last_error JSONB,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 19. knowledge_graph_edges [NEW]
CREATE TABLE knowledge_graph_edges (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES accounts(id),
  source_type TEXT NOT NULL,                -- 'person', 'organization', 'project', 'goal', 'event', 'topic'
  source_id TEXT NOT NULL,                  -- entity identifier
  source_label TEXT NOT NULL,               -- human-readable name
  relationship TEXT NOT NULL,               -- 'works_at', 'reports_to', 'collaborates_on', 'depends_on', 'related_to'
  target_type TEXT NOT NULL,
  target_id TEXT NOT NULL,
  target_label TEXT NOT NULL,
  weight FLOAT DEFAULT 1.0,
  metadata JSONB DEFAULT '{}',
  confidence FLOAT DEFAULT 0.8,
  source_file TEXT,                         -- which knowledge file owns this edge
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_kg_user_source ON knowledge_graph_edges(user_id, source_type, source_id);
CREATE INDEX idx_kg_user_target ON knowledge_graph_edges(user_id, target_type, target_id);
CREATE INDEX idx_kg_relationship ON knowledge_graph_edges(user_id, relationship);

-- RLS on ALL tables
ALTER TABLE accounts ENABLE ROW LEVEL SECURITY;
ALTER TABLE channel_connections ENABLE ROW LEVEL SECURITY;
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE runs ENABLE ROW LEVEL SECURITY;
ALTER TABLE memories ENABLE ROW LEVEL SECURITY;
ALTER TABLE tool_executions ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE oauth_tokens ENABLE ROW LEVEL SECURITY;
ALTER TABLE proactive_triggers ENABLE ROW LEVEL SECURITY;
ALTER TABLE sync_cursors ENABLE ROW LEVEL SECURITY;
ALTER TABLE side_effects ENABLE ROW LEVEL SECURITY;
ALTER TABLE knowledge_files ENABLE ROW LEVEL SECURITY;
ALTER TABLE profiling_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE behavioral_rules ENABLE ROW LEVEL SECURITY;
ALTER TABLE feedback_signals ENABLE ROW LEVEL SECURITY;
ALTER TABLE goals ENABLE ROW LEVEL SECURITY;
ALTER TABLE delegations ENABLE ROW LEVEL SECURITY;
ALTER TABLE research_jobs ENABLE ROW LEVEL SECURITY;
ALTER TABLE workflows ENABLE ROW LEVEL SECURITY;
ALTER TABLE knowledge_graph_edges ENABLE ROW LEVEL SECURITY;
```

4. Pydantic Contracts — All Data Models

4.1 Core Message Models

```
class InboundMessage(BaseModel):
    channel: ChannelType
    channel_identifier: str
    content: str
    input_modality: InputModality = InputModality.TEXT
    media_url: Optional[str] = None          # S3 URL for voice/image/doc
    media_type: Optional[str] = None         # mime type
    wa_message_id: Optional[str] = None
    reply_to_id: Optional[str] = None
    timestamp: datetime

class ProcessedMessage(BaseModel):           # NEW: output of multi-modal pipeline
    original: InboundMessage
    normalized_text: str                     # always text after processing
    modality: InputModality
    transcription_confidence: Optional[float] = None
    extracted_entities: dict = {}             # contacts, dates, amounts, action items
    emotion_detected: EmotionState = EmotionState.NEUTRAL
    content_provenance: ContentProvenance = ContentProvenance.USER_DIRECT

class OutboundMessage(BaseModel):
    channel: ChannelType
    recipient_id: str
    content: str
    output_modality: OutputModality = OutputModality.TEXT  # NEW: text, voice, document
    attachment_url: Optional[str] = None      # for docs/images/audio
    reply_to_id: Optional[str] = None
    buttons: Optional[list[WhatsAppButton]] = None
    metadata: dict = {}
```

4.2 Intent Classification & Tier Routing

```
class IntentClassification(BaseModel):
    intent: str
    tier: Tier
    confidence: float = Field(ge=0.0, le=1.0)
    required_tools: list[str] = []
    is_continuation: bool = False
    continuation_run_id: Optional[UUID] = None
    entities: dict = {}
    requires_delegation: bool = False          # NEW
    requires_research: bool = False            # NEW
    emotion_context: EmotionState = EmotionState.NEUTRAL  # NEW

class TierRoutingConfig(BaseModel):
    intent_pattern: str
    default_tier: Tier
    escalation_rules: dict = {}
    required_tools: list[str] = []
    max_cost_cents: int = 50
    preferred_provider: Optional[LLMProvider] = None  # NEW
```

4.3 Tool Contracts (Enhanced with MCP)

```
class ToolSpec(BaseModel):
    name: str
    description: str
    input_schema: dict
    output_schema: dict
```

```
    risk_level: RiskLevel = RiskLevel.NONE
    is_reversible: bool = False
    requires_approval_above: RiskLevel = RiskLevel.MEDIUM
    timeout_ms: int = 10000
    retry_policy: dict = {"max_retries": 2, "backoff": "exponential"}
    rate_limit_per_min: int = 30
    tags: list[str] = []
    is_mcp: bool = False                      # NEW
    mcp_server_id: Optional[str] = None       # NEW
    capability_scope: list[str] = []          # NEW: required capabilities

class ToolCall(BaseModel):
    tool_name: str
    arguments: dict
    idempotency_key: str
    envelope_id: UUID
    run_id: UUID
    input_provenance: ContentProvenance = ContentProvenance.USER_DIRECT  # NEW
    required_capabilities: list[str] = []     # NEW

class ToolResult(BaseModel):
    tool_name: str
    status: str
    output: Optional[dict] = None
    error: Optional[dict] = None
    latency_ms: int
    cost_cents: int = 0
    compensating_action: Optional[dict] = None  # NEW: for saga rollback
```

4.4 LLM Router Contracts [NEW]

```
class LLMRequest(BaseModel):
    model_preference: Optional[LLMProvider] = None  # hint, not requirement
    messages: list[dict]
    tools: Optional[list[dict]] = None
    temperature: float = 0.7
    max_tokens: int = 2000
    structured_output: Optional[dict] = None    # JSON schema for structured output
    task_type: str = "general"                   # classification, drafting, analysis, extraction
    max_cost_cents: float = 10.0
    max_latency_ms: int = 15000
    pii_content: bool = False                    # if true, route to local model
    requires_safety_check: bool = False          # if true, add secondary validation

class LLMResponse(BaseModel):
    provider: LLMProvider
    model: str
    content: str
    tool_calls: Optional[list[dict]] = None
    usage: TokenUsage
    cost_cents: float
    latency_ms: int
    was_failover: bool = False                   # true if primary was unavailable
    safety_validated: bool = False

class ProviderHealth(BaseModel):
    provider: LLMProvider
    is_healthy: bool
    latency_p50_ms: int
    latency_p95_ms: int
    error_rate_1h: float
    last_error_at: Optional[datetime] = None
    cost_multiplier: float = 1.0                # relative cost vs baseline
```

4.5 Delegation Contracts [NEW]

```
class DelegationRequest(BaseModel):
    delegate_name: str
    delegate_contact: Optional[str] = None      # resolved from TEAM.md or contacts
    task_description: str
    context: dict = {}
    deadline: Optional[datetime] = None
    priority: RiskLevel = RiskLevel.MEDIUM
    reminder_schedule: list[datetime] = []      # auto-generated if not provided
    completion_criteria: Optional[str] = None
    notify_channel: Optional[ChannelType] = None

class DelegationUpdate(BaseModel):
    delegation_id: UUID
    status: DelegationStatus
    result: Optional[str] = None
    updated_by: str                             # 'user', 'delegate', 'system'
```

4.6 Workflow Contracts [NEW]

```
class WorkflowDefinition(BaseModel):
    name: str
    description: Optional[str] = None
    trigger: WorkflowTrigger
    actions: list[WorkflowAction]
    conditions: Optional[list[WorkflowCondition]] = None

class WorkflowTrigger(BaseModel):
    type: WorkflowTriggerType                   # schedule, event, condition, manual
    config: dict                                # cron expression, event match, etc.

class WorkflowAction(BaseModel):
    step: int
    tool_name: str
    arguments_template: dict                    # supports {{variables}} from trigger context
    on_failure: str = "stop"                    # stop, skip, retry
    approval_required: bool = False

class WorkflowCondition(BaseModel):
    field: str
    operator: str                               # eq, ne, gt, lt, contains, matches
    value: str
```

5. Unified API Specification — All Endpoints

5.1 Gateway API

Endpoint	Method	Purpose	Request/Response
/webhook/whatsapp	POST	WhatsApp Business API webhook	Validates HMAC-SHA256, dedup, routes to Brain
/webhook/imessage	POST	Apple Messages for Business webhook	Validates cert chain, routes to Brain
/webhook/slack	POST	Slack Events API webhook	Validates Slack signature, routes to Brain
/api/v1/message	POST	Direct API message (web/test)	{channel, content, modality} → {run_id, status}
/api/v1/stream/{run_id}	GET (SSE)	Stream progress for long-running tasks [NEW]	Server-Sent Events: {step, status, partial_result}
/api/v1/voice/transcribe	POST	Direct voice transcription [NEW]	{audio_url} → {text, confidence, language}

5.2 Core Agent API

Endpoint	Method	Purpose	Request/Response
/api/v1/runs/{id}	GET	Get run status and result	{state, tier, result, cost, latency}
/api/v1/runs/{id}/approve	POST	Approve a pending action	{approval: approved|denied} → executes or cancels
/api/v1/memories/search	POST	Semantic memory search	{query, limit} → [{content, similarity, type}]
/api/v1/connectors	GET	List connected services	[{provider, status, scopes, last_sync}]
/api/v1/connectors/{provider}/auth	GET	Start OAuth flow	Redirect to provider OAuth

5.3 Behavioral Intelligence API

Endpoint	Method	Purpose	Request/Response
/api/v1/knowledge/{file_path}	GET	Get current version of knowledge file	{content, version, completeness, updated_at}
/api/v1/knowledge/{file_path}	PUT	User edits a knowledge file directly	{content} → validates, stores new version
/api/v1/knowledge	GET	List all knowledge files with completeness	[{file_path, layer, completeness, token_count}]
/api/v1/knowledge/review	POST	Trigger self-review and gap detection	{gaps[], suggestions[], questions[]}
/api/v1/knowledge/graph	GET	Query knowledge graph [NEW]	{entity_type, entity_id} → {edges[], related_entities[]}
/api/v1/profiling/next	GET	Get next profiling session/question	{dimension, question, progress_pct} or null

Endpoint	Method	Purpose	Request/Response
/api/v1/profiling/answer	POST	Submit answer to profiling question	{session_id, answer} → next question
/api/v1/goals	GET/POST	List or create goals	CRUD for goals table
/api/v1/goals/{id}	PUT/DELETE	Update or remove a goal	Standard REST
/api/v1/heartbeat	GET	Get current HEARTBEAT.md rendered	{content, goals[], projects[]}
/api/v1/feedback	POST	Submit correction/feedback signal	{run_id, signal_type, original, corrected}
/api/v1/rules	GET	List all behavioral rules	[{category, rule_key, rule_value, source, confidence}]
/api/v1/rules/{id}	PUT/DELETE	User edits or removes a behavioral rule	Standard REST

5.4 New v4.0 APIs

Endpoint	Method	Purpose	Request/Response
/api/v1/delegations	GET/POST	List or create delegation [NEW]	{delegate_name, task, deadline} → {delegation_id}
/api/v1/delegations/{id}	GET/PUT	Check or update delegation status [NEW]	{status, result, reminders_sent}
/api/v1/research	GET/POST	List or create research job [NEW]	{query, sources, schedule} → {job_id}
/api/v1/research/{id}	GET/PUT/DELETE	Manage research job [NEW]	{findings[], next_run, total_cost}
/api/v1/workflows	GET/POST	List or create workflow [NEW]	{name, trigger, actions} → {workflow_id}
/api/v1/workflows/{id}	GET/PUT/DELETE	Manage workflow [NEW]	{status, run_count, last_run, next_run}
/api/v1/workflows/{id}/test	POST	Dry-run a workflow [NEW]	{simulated_results[], estimated_cost}
/api/v1/team	GET	Get team from TEAM.md [NEW]	[{name, role, contact, preferences}]
/api/v1/export	GET	Export all user data (GDPR) [NEW]	ZIP: knowledge files, rules, memories, logs

5.5 Internal/Admin API

Endpoint	Method	Purpose
/internal/health	GET	Shallow health check (process alive)
/internal/health/deep	GET	Deep health: Postgres, Redis, all LLM providers, each connector

Endpoint	Method	Purpose
/internal/metrics	GET	Prometheus-format metrics for observability
/internal/runs/{id}/replay	POST	Replay a run from event log (debugging)
/internal/cache/flush	POST	Flush semantic cache for a user (admin)
/internal/triggers/fire	POST	Manually fire a proactive trigger (testing)
/internal/knowledge/consolidate	POST	Manually trigger nightly consolidation (testing)
/internal/knowledge/review	POST	Manually trigger weekly self-review (testing)
/internal/llm/health	GET	Check all LLM provider health + failover status [NEW]
/internal/llm/route-test	POST	Test LLM routing decision for a given request [NEW]
/internal/experiments	GET/POST	A/B test management [NEW]

7. Intelligence Engine — 4-Tier Reasoning + Hierarchical Decomposition

The reasoning engine matches effort to task complexity. The key enhancement in v4.0: every tier now receives personalized context from the knowledge file system, uses the multi-provider LLM router for optimal model selection, and Tier 2-3 use **hierarchical task decomposition** with adaptive iteration limits instead of fixed caps.

7.1 Tier Definitions & Processing Pipelines

Tier 0: Instant (15-20% of traffic)

Pattern-matched responses with zero LLM calls. Now personalized via IDENTITY.md variables. 1. Regex/keyword match → 2. Lookup response template → 3. Variable substitution (user name, time, IDENTITY.md data) → 4. Return immediately.

Cost: < $0.001 | **Example:** 'Good morning' → personalized greeting with today's agenda summary.

Tier 1: Single-Shot (50-60% of traffic)

One LLM call with function calling. Context includes SOUL.md personality + relevant AGENTS.md rules. Model selected by LLM Router based on task type (default: GPT-4o-mini). 1. Context compile (3,400 tokens max including ~900 from knowledge files) → 2. LLM Router selects model → 3. Single inference with tool schemas → 4. Execute tool call → 5. Format and return.

Cost: $0.002-$0.008 | **Latency:** 1-3s | **Example:** 'What's on my calendar tomorrow?'

Tier 2: ReAct Loop with Hierarchical Decomposition (20-25% of traffic)

Iterative Thought-Action-Observation with GPT-4o (or Claude Sonnet for communication tasks). Full user context including USER.md, HEARTBEAT.md goals, TEAM.md, and relevant AGENTS.md rules. **v4.0 change:** Adaptive iteration limit replaces fixed max-5. Self-reflection after completion.

```
async def execute_tier2(classification: IntentClassification, context: CompiledContext) -> RunResult:
    # 1. Context compile (11,000 tokens max including ~2,800 from knowledge files)
    # 2. Select provider via LLM Router
    provider = await llm_router.select(task_type=classification.intent, context_size=context.token_count)

    max_iterations = compute_adaptive_limit(classification, context)  # NEW: value-of-information calc
    iterations = 0

    while iterations < max_iterations:
        # 3. Thought + Action
        response = await llm_router.call(provider, context.messages, tools=context.tool_schemas)

        if response.tool_calls:
            for tool_call in response.tool_calls:
                # 4. Semantic validation before execution (NEW)
                validation = await validate_tool_call(tool_call, context.knowledge_files)
                if not validation.is_valid:
                    context.add_observation(f"Tool call rejected: {validation.reason}")
                    continue

                # 5. Execute tool, get Observation
                result = await execute_tool(tool_call, provenance=context.input_provenance)
                context.add_observation(result)
        else:
            break  # No more tool calls needed

        iterations += 1
```

```
        # NEW: Check if marginal value of next iteration justifies cost
        if not should_continue(context, iterations, max_iterations):
            break

    # 6. Self-reflection (NEW): Did we answer the underlying question?
    reflection = await self_reflect(classification.intent, context.result, provider)
    if reflection.needs_followup:
        context.add_followup_suggestion(reflection.suggestion)

    return format_response(context)

def compute_adaptive_limit(classification, context) -> int:
    """Value-of-information based iteration limit"""
    base = 3
    if classification.tier == Tier.REACT:
        if len(classification.required_tools) > 2: base = 5
        if 'research' in classification.intent: base = 7
        if context.has_delegation: base = 4
    return min(base, 10)  # hard cap at 10
```

Tier 3: Multi-Step Plan with MCTS Planning (3-5% of traffic)

Full plan generation with critic review and checkpointed execution. Maximum knowledge file injection for true chief-of-staff behavior. **v4.0:** Uses Monte Carlo Tree Search for high-stakes planning, saga pattern for compensation, and parallel sub-task execution.

```
async def execute_tier3(classification: IntentClassification, context: CompiledContext) -> RunResult:
    # 1. Context compile (23,500 tokens max including ~5,500 from knowledge files)

    # 2. Hierarchical decomposition (NEW)
    subtasks = await decompose_task(classification.intent, context)
    # Each subtask gets its own tier assignment
    # Example: "Prepare for board meeting" decomposes into:
    #   T1: Pull calendar details for the meeting
    #   T2: Search emails for relevant threads
    #   T2: Draft talking points for each agenda item
    #   T3: Identify tough questions and prepare answers
    #   T1: Create executive summary document

    # 3. Plan generation with MCTS (NEW for high-stakes)
    if classification.is_high_stakes:
        plans = await mcts_plan(subtasks, context, num_rollouts=5)
        plan = select_best_plan(plans, context.agents_rules)
    else:
        plan = await generate_plan(subtasks, context)

    # 4. Critic review
    critique = await critic_review(plan, context)
    if critique.major_issues:
        plan = await revise_plan(plan, critique, context)

    # 5. Checkpointed execution with saga compensation (NEW)
    results = []
    for step in plan.steps:
        checkpoint = await save_checkpoint(step)
        try:
            result = await execute_step(step, context)
            results.append(result)
        except Exception as e:
            # Saga compensation: roll back completed steps
            await compensate_completed_steps(results)
            # Partial delivery (NEW): tell user what succeeded
            return partial_result(results, failed_step=step, error=e)

    # 6. Re-plan on failure (max 3 re-plans)
    # ... existing logic enhanced with subtask-level retry
```

```
    return format_response(results)
```

Cost: $0.10-$0.30 | **Latency:** 10-60s | **Example:** 'Prepare for my investor meeting on Thursday' → full briefing workflow

8. Knowledge File System — 9 Per-User File Templates

Each user gets **9 structured markdown files** (up from 7 in v3.2) that collectively represent a complete model of who the user is, how they work, what they're trying to achieve, and how the agent should behave. Files are versioned in PostgreSQL with S3 snapshots. Redis hot cache for IDENTITY.md, SOUL.md, and AGENTS.md. **New in v4.0:** TEAM.md and WORKFLOWS.md added.

File	Layer	Purpose	Key Sections	Injection Tiers
USER.md	Brain	Who the user is: demographics, preferences, work context, relationships, energy patterns	Basics, Work, People, Operations, Communication, Energy, Friction Points	T1 (reduced), T2-T3 (full)
SOUL.md	Brain	Agent personality: character archetype, tone, style, emotional texture, humor calibration	Character, Voice, Emotional Range, Humor, Boundaries, Adaptation Rules	T1-T3 (always)
IDENTITY.md	Brain	Agent identity: name, vibe, energy level, greeting style. Used for T0 template vars	Name, Vibe, Energy, Greeting Template, Signature	T0-T3 (always)
AGENTS.md	DNA	Operating rules: decision protocols, risk tolerance, autonomy levels, escalation rules, delegation rules	Autonomy, Escalation, Risk, Boundaries, Initiative, Communication Rules, Delegation Preferences, Research Preferences	T1-T3 (relevant sections)
MEMORY.md	DNA	Memory management: what to remember, retention rules, pruning logic, knowledge compounding	Memory Architecture, Retention Rules, Knowledge Compounding, Daily Log Template, Contradiction Resolution	T3 only
HEARTBEAT.md	Brain	Living pulse: active goals, projects, deadlines, milestones, blockers, idea capture	This Week, This Month, This Quarter, Long-term, Idea Capture, Blocked Items, Delegation Tracker	T2-T3
TOOLS.md	Bones/Muscles	Tool preferences: which tools, model routing, cost preferences, MCP server catalog	Connected Services, Tool Preferences, Model Preferences, Cost Limits, MCP Servers	T2-T3 (relevant)
TEAM.md [NEW]	Brain	Team map: direct reports, key stakeholders, working styles, reliability, delegation history	Direct Reports, Key Stakeholders, Decision Authority, Communication Preferences per Person, Delegation Track Record	T2-T3 (when team-related)
WORKFLOWS.md [NEW]	DNA	User-defined automations: triggers, conditions, actions, schedules, templates	Active Workflows, Workflow Templates, Trigger Definitions, Action Library, Error Handling Preferences	On workflow trigger

8.1 USER.md Template

```
# USER.md -- Deep User Profile
# Generated by Brain Layer | Last updated: {timestamp}
```

```
## BASICS
### Identity
{name, age_range, location, timezone}
### Work
{role, company, industry, team_size, reporting_structure}
### Background
{education, career_history, expertise_areas}

## PEOPLE
### Key Relationships
{name: relationship, communication_preference, interaction_frequency, notes}
### Contact Graph Summary
{top contacts by interaction frequency, organizational clusters}

## OPERATIONS
### Daily Patterns
{wake_time, work_start, lunch_preference, work_end, wind_down}
### Meeting Preferences
{preferred_times, max_per_day, buffer_between, prep_time_needed}
### Communication Preferences
{email_style, message_length, formality_by_context}

## ENERGY & PRODUCTIVITY
### Peak Hours
{high_energy_times, low_energy_times, creative_times}
### Friction Points
{known_frustrations, pet_peeves, things_that_waste_time}

## INTERESTS & VALUES
{personal_interests, professional_values, decision_making_style}
```

8.8 TEAM.md Template [NEW]

```
# TEAM.md -- Team Knowledge Map
# Generated by Brain Layer | Last updated: {timestamp}

## DIRECT REPORTS
### {Person Name}
- Role: {title}
- Contact: {email, slack, phone}
- Working Style: {detail-oriented, fast-mover, needs-context, etc.}
- Strengths: {areas of excellence}
- Growth Areas: {areas needing support}
- Preferred Communication: {slack DM, email, quick call}
- Reliability Score: {high/medium/low} -- based on delegation history
- Current Load: {light/moderate/heavy}
- Notes: {any special considerations}

## KEY STAKEHOLDERS
### {Stakeholder Name}
- Role: {title, organization}
- Relationship: {manager, peer, client, investor, board_member}
- Communication Style: {formal, casual, data-driven, story-driven}
- Decision Authority: {what they can approve/block}
- Preferred Meeting Format: {in-person, video, async}
- Notes: {political considerations, historical context}

## ORGANIZATIONAL CONTEXT
### Decision Authority Map
{domain: who_approves, escalation_path}
### Team Norms
{meeting cadence, communication channels, documentation expectations}

## DELEGATION HISTORY
### Recent Delegations
{person: task, outcome, on_time, quality_rating}
### Delegation Patterns
```

```
{who gets what types of tasks, success rates, common issues}
```

8.9 WORKFLOWS.md Template [NEW]

```
# WORKFLOWS.md -- User-Defined Automations
# Generated by DNA Layer | Last updated: {timestamp}

## ACTIVE WORKFLOWS
### {Workflow Name}
- Trigger: {schedule: "0 8 * * 1" | event: "email_received" | condition: "..."}
- Steps:
  1. {action_description} → {tool_name}({params})
  2. {action_description} → {tool_name}({params})
- On Failure: {stop | skip | retry | notify_user}
- Last Run: {timestamp}
- Status: {active | paused}

## WORKFLOW TEMPLATES
### Weekly Report
- Trigger: Every Monday 8am
- Steps: Pull metrics → Summarize changes → Send to channel
### Urgent Email Forward
- Trigger: Email with subject containing "urgent"
- Steps: Extract summary → Forward to WhatsApp
### Post-Meeting Follow-up
- Trigger: Calendar event ends
- Condition: Event has external attendees
- Steps: Draft follow-up email → Queue for review

## ERROR HANDLING PREFERENCES
{retry_policy, notification_preferences, fallback_actions}
```

9. Multi-Provider LLM Router — Model-Agnostic Brain [NEW]

The LLM Router is the single most important v4.0 addition. It provides a unified interface to multiple LLM providers, enabling automatic failover, task-optimized routing, cost optimization, and elimination of single-provider dependency. Every LLM call in the system goes through this router.

9.1 Provider Configuration

```
PROVIDER_CONFIG = {
    "openai": {
        "models": {
            "gpt-4o": {"cost_per_1k_input": 0.0025, "cost_per_1k_output": 0.01, "max_context": 128000},
            "gpt-4o-mini": {"cost_per_1k_input": 0.00015, "cost_per_1k_output": 0.0006, "max_context": 128000},
        },
        "strengths": ["function_calling", "structured_output", "speed"],
        "health_endpoint": "https://api.openai.com/v1/models",
    },
    "anthropic": {
        "models": {
            "claude-sonnet-4-20250514": {"cost_per_1k_input": 0.003, "cost_per_1k_output": 0.015, "max_context": 200000}
        },
        "strengths": ["nuanced_writing", "safety", "long_context", "instruction_following"],
        "health_endpoint": "https://api.anthropic.com/v1/messages",
    },
    "google": {
        "models": {
            "gemini-2.0-flash": {"cost_per_1k_input": 0.0001, "cost_per_1k_output": 0.0004, "max_context": 1000000},
            "gemini-2.0-pro": {"cost_per_1k_input": 0.00125, "cost_per_1k_output": 0.005, "max_context": 1000000},
        },
        "strengths": ["long_context", "multimodal", "cost_efficiency", "bulk_processing"],
        "health_endpoint": "https://generativelanguage.googleapis.com/v1/models",
    },
    "local": {
        "models": {
            "llama-4-8b": {"cost_per_1k_input": 0.00005, "cost_per_1k_output": 0.0001, "max_context": 128000},
        },
        "strengths": ["pii_safety", "air_gapped", "zero_latency_network"],
        "health_endpoint": "http://localhost:8000/health",
    },
}
```

9.2 Task-Optimized Routing

```
TASK_ROUTING_TABLE = {
    # Task type → preferred provider, fallback chain
    "intent_classification": {"primary": "openai:gpt-4o-mini", "fallback": ["google:gemini-2.0-flash", "local:llama-4-8b
    "single_tool_call":      {"primary": "openai:gpt-4o-mini", "fallback": ["anthropic:claude-sonnet-4-20250514"]},
    "email_drafting":        {"primary": "anthropic:claude-sonnet-4-20250514", "fallback": ["openai:gpt-4o"]},
    "complex_reasoning":     {"primary": "openai:gpt-4o", "fallback": ["anthropic:claude-sonnet-4-20250514"]},
    "long_document":         {"primary": "google:gemini-2.0-pro", "fallback": ["anthropic:claude-sonnet-4-20250514"]},
    "knowledge_extraction":  {"primary": "openai:gpt-4o-mini", "fallback": ["google:gemini-2.0-flash"]},
    "safety_validation":     {"primary": "anthropic:claude-sonnet-4-20250514", "fallback": ["openai:gpt-4o"]},
    "pii_processing":        {"primary": "local:llama-4-8b", "fallback": ["openai:gpt-4o"]},  # local first for PII
    "bulk_processing":       {"primary": "google:gemini-2.0-flash", "fallback": ["openai:gpt-4o-mini"]},
    "research_synthesis":    {"primary": "openai:gpt-4o", "fallback": ["anthropic:claude-sonnet-4-20250514"]},
}
```

9.3 Automatic Failover

```
class LLMRouter:
    async def call(self, request: LLMRequest) -> LLMResponse:
        routing = self.get_routing(request.task_type)

        # Check provider health
        for model_spec in [routing["primary"]] + routing["fallback"]:
            provider, model = model_spec.split(":")
            health = await self.get_provider_health(provider)

            if not health.is_healthy:
                log.warning(f"Provider {provider} unhealthy, skipping")
                continue

            if request.pii_content and provider != "local":
                continue  # Skip non-local for PII

            try:
                response = await self._call_provider(provider, model, request)

                # Track cost and latency
                await self.track_usage(provider, model, response)

                return LLMResponse(
                    provider=provider,
                    model=model,
                    content=response.content,
                    tool_calls=response.tool_calls,
                    usage=response.usage,
                    cost_cents=self.compute_cost(provider, model, response.usage),
                    latency_ms=response.latency_ms,
                    was_failover=(model_spec != routing["primary"]),
                )
            except (TimeoutError, APIError, RateLimitError) as e:
                log.error(f"Provider {provider} failed: {e}")
                await self.mark_unhealthy(provider, duration_s=60)
                continue

        raise AllProvidersFailedError("No LLM provider available")

    async def get_provider_health(self, provider: str) -> ProviderHealth:
        """Check health from Redis cache, updated every 30s by background job"""
        cached = await redis.get(f"llm_health:{provider}")
        if cached:
            return ProviderHealth.parse_raw(cached)
        return await self._probe_provider(provider)
```

21. Multi-Modal Pipeline — Voice, Image, Document [NEW]

21.1 Voice Processing Pipeline

```
async def process_voice_message(message: InboundMessage) -> ProcessedMessage:
    """Convert voice note to text with metadata extraction"""
    # 1. Download audio from WhatsApp/iMessage
    audio_bytes = await download_media(message.media_url)
    audio_path = await save_to_s3(audio_bytes, f"voice/{message.id}.ogg")

    # 2. Transcribe with Whisper
    transcription = await openai.audio.transcriptions.create(
        model="whisper-1",
        file=audio_bytes,
        response_format="verbose_json",
        timestamp_granularities=["segment"]
    )

    # 3. Extract intent signals from audio metadata
    metadata = {
        "duration_seconds": transcription.duration,
        "language": transcription.language,
        "segments": [{"text": s.text, "start": s.start, "end": s.end} for s in transcription.segments],
    }

    # 4. Detect emotion from speech patterns (speaking speed, pauses)
    avg_words_per_second = len(transcription.text.split()) / max(transcription.duration, 1)
    emotion = EmotionState.RUSHED if avg_words_per_second > 3.5 else EmotionState.NEUTRAL

    return ProcessedMessage(
        original=message,
        normalized_text=transcription.text,
        modality=InputModality.VOICE,
        transcription_confidence=sum(s.get("avg_logprob", -0.5) for s in transcription.segments) / max(len(transcription
        extracted_entities=metadata,
        emotion_detected=emotion,
    )
```

21.2 Image Processing Pipeline

```
async def process_image_message(message: InboundMessage) -> ProcessedMessage:
    """Process image through vision model for entity extraction"""
    # 1. Download and store image
    image_bytes = await download_media(message.media_url)
    image_url = await save_to_s3(image_bytes, f"images/{message.id}.jpg")

    # 2. Classify image type
    classification = await llm_router.call(LLMRequest(
        task_type="intent_classification",
        messages=[{
            "role": "user",
            "content": [
                {"type": "image_url", "image_url": {"url": image_url}},
                {"type": "text", "text": "Classify this image: business_card, receipt, whiteboard, screenshot, document,
            ]
        }],
        structured_output={"type": "string", "confidence": "number"},
    ))

    image_type = classification.parsed["type"]

    # 3. Type-specific extraction
```

```
    extraction_prompts = {
        "business_card": "Extract: name, title, company, email, phone, address. Return structured JSON.",
        "receipt": "Extract: merchant, date, items with prices, subtotal, tax, total, payment_method. Return JSON.",
        "whiteboard": "Extract: all text, diagrams as descriptions, action items, key decisions. Return JSON.",
        "screenshot": "Describe what's shown. Extract any error messages, data values, or actionable information. Return
        "document": "Summarize the document. Extract key points, dates, names, action items. Return JSON.",
    }

    entities = await llm_router.call(LLMRequest(
        task_type="knowledge_extraction",
        messages=[{
            "role": "user",
            "content": [
                {"type": "image_url", "image_url": {"url": image_url}},
                {"type": "text", "text": extraction_prompts.get(image_type, extraction_prompts["screenshot"])}
            ]
        }],
    ))

    # 4. Auto-action based on type
    action_text = f"[Image: {image_type}] {entities.content}"
    if image_type == "business_card":
        action_text += "\nWould you like me to create a contact from this business card?"

    return ProcessedMessage(
        original=message,
        normalized_text=action_text,
        modality=InputModality.IMAGE,
        extracted_entities={"image_type": image_type, "extraction": entities.parsed},
    )
```

21.3 Voice Output Pipeline

```
async def generate_voice_response(text: str, user: Account) -> str:
    """Convert text response to audio for voice-preferred delivery"""
    # Use OpenAI TTS or ElevenLabs based on TOOLS.md preference
    tts_provider = user.tools_preferences.get("tts_provider", "openai")

    if tts_provider == "openai":
        response = await openai.audio.speech.create(
            model="tts-1",
            voice=user.tools_preferences.get("tts_voice", "nova"),
            input=text[:4096],  # TTS limit
        )
        audio_bytes = response.content

    audio_url = await save_to_s3(audio_bytes, f"voice_out/{uuid4()}.mp3")
    return audio_url
```

24. Team Awareness & Delegation Engine [NEW]

The Delegation Engine transforms the agent from a personal tool into an organizational multiplier. It manages task delegation to team members, tracks completion, sends reminders, and escalates overdue items. All team knowledge lives in TEAM.md and the delegations table.

24.1 Delegation Flow

```
async def create_delegation(user_id: UUID, request: DelegationRequest) -> Delegation:
    # 1. Resolve delegate contact from TEAM.md or contacts
    team_md = await get_cached_file(user_id, "TEAM.md")
    delegate = resolve_delegate(request.delegate_name, team_md)

    if not delegate:
        return ask_user(f"I don't have contact info for {request.delegate_name}. "
                        f"How should I reach them?")

    # 2. Generate context-appropriate message
    message = await llm_router.call(LLMRequest(
        task_type="email_drafting",
        messages=[{
            "role": "system",
            "content": f"Draft a delegation message from {user.display_name} to {delegate.name}. "
                       f"Tone: {delegate.communication_preference}. "
                       f"Task: {request.task_description}. "
                       f"Deadline: {request.deadline}. "
                       f"Context: {request.context}"
        }],
    ))

    # 3. Create delegation record
    delegation = await db.execute(
        INSERT INTO delegations (user_id, delegate_name, delegate_contact, delegate_channel,
            task_description, context, deadline, reminder_schedule, completion_criteria)
        VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9) RETURNING *
    )

    # 4. Send via appropriate channel
    await send_delegation_message(delegate, message, delegation.id)

    # 5. Schedule reminders
    for reminder_time in compute_reminder_schedule(request.deadline):
        await schedule_trigger(user_id, "delegation", {
            "delegation_id": delegation.id,
            "type": "reminder",
        }, fire_at=reminder_time)

    # 6. Update HEARTBEAT.md
    await update_knowledge_file(user_id, "HEARTBEAT.md", "Delegation Tracker",
        f"{delegate.name}: {request.task_description}", 0.95)

    return delegation
```

24.2 Multi-Stakeholder Scheduling

```
async def schedule_multi_person(user_id: UUID, participants: list[str],
                                duration_min: int, constraints: dict) -> list[TimeSlot]:
    # 1. Resolve participants from TEAM.md + contacts
    team_md = await get_cached_file(user_id, "TEAM.md")
    resolved = [resolve_person(p, team_md) for p in participants]

    # 2. Fetch calendars (user + any connected calendars for team members)
    availability = {}
```

```
for person in resolved:
    if person.has_connected_calendar:
        cal = await google_calendar.get_free_busy(person.calendar_id, range_days=14)
    else:
        # Use known patterns from TEAM.md
        cal = estimate_availability(person, team_md)
    availability[person.name] = cal

# 3. Apply constraints from USER.md and TEAM.md
# - User's preferred meeting times
# - Timezone handling for distributed teams
# - Buffer between meetings
# - Person-specific preferences (e.g., "Sarah prefers mornings")
constraints = merge_constraints(
    user_constraints=get_user_meeting_prefs(user_id),
    person_constraints={p.name: p.meeting_preferences for p in resolved},
    explicit_constraints=constraints,
)

# 4. Find overlapping free slots
slots = find_common_availability(availability, duration_min, constraints)

# 5. Rank by preference alignment
ranked = rank_slots(slots, constraints, user_timezone=user.timezone)

return ranked[:5]  # Top 5 options
```

25. Autonomous Research & Monitoring Engine [NEW]

The Research Engine enables standing research orders that run autonomously on schedules. Examples: 'Monitor competitors weekly', 'Track SEC filings', 'Research market trends monthly'. Research jobs are Temporal workflows with persistent findings accumulation.

25.1 Research Job Execution

```
@temporal_workflow
async def execute_research_job(job_id: UUID):
    job = await db.fetch("SELECT * FROM research_jobs WHERE id = $1", job_id)
    user = await get_user(job.user_id)

    # 1. Generate search queries from research description
    queries = await llm_router.call(LLMRequest(
        task_type="complex_reasoning",
        messages=[{
            "role": "system",
            "content": f"Generate 3-5 search queries to research: {job.query}. "
                       f"Previous findings: {json.dumps(job.findings[-5:])}. "
                       f"Focus on NEW information since last run."
        }],
    ))

    # 2. Execute searches with cost tracking
    results = []
    cost = 0
    for query in queries.parsed:
        if cost >= job.max_cost_per_run:
            break
        search_results = await tavily.search(query, max_results=5)

        # 3. Fetch full pages for top results
        for result in search_results[:3]:
            page_content = await fetch_page(result.url)
            results.append({"url": result.url, "title": result.title, "content": page_content[:5000]})
            cost += 0.01

    # 4. Synthesize findings (deduplicate against previous)
    synthesis = await llm_router.call(LLMRequest(
        task_type="research_synthesis",
        messages=[{
            "role": "system",
            "content": f"Synthesize these research results about: {job.query}\n"
                       f"Previous findings (do NOT repeat): {json.dumps(job.findings[-10:])}\n"
                       f"New results: {json.dumps(results)}\n"
                       f"Format: {job.delivery_format}. "
                       f"Only include genuinely new information."
        }],
    ))

    # 5. Store findings
    new_findings = {"date": now().isoformat(), "summary": synthesis.content, "sources": [r["url"] for r in results]}
    await db.execute("UPDATE research_jobs SET findings = findings || $1, last_run_at = now(), total_cost = total_cost +
                     json.dumps([new_findings]), cost, job_id)

    # 6. Deliver via proactive trigger
    await fire_proactive(job.user_id, "research", {
        "job_id": job_id,
        "title": job.title,
        "summary": synthesis.content,
    }, channel=job.delivery_channel)
```

32. Security Implementation — Privilege Isolation [ENHANCED]

v4.0 fundamentally upgrades security from pattern-matching input sanitization to a **privilege-isolated, capability-based security model**. This is non-negotiable for a system that reads untrusted email content, processes web search results, and has the ability to send emails and manage calendars on the user's behalf.

Control	Implementation	When	v4.0 Changes
Input sanitization	Strip prompt injection patterns from messages before LLM context	Every inbound message	Now defense-in-depth layer, not sole defense
Content provenance	Every text chunk in LLM context tagged with origin: user_direct, email_body, web_search, calendar_desc, document, mcp_result	Every LLM call	NEW — enables privilege isolation
Privilege isolation	External content (emails, web, calendar) processed in sandboxed context with REDUCED tool permissions. Cannot invoke send_email or create_event in same context window.	T2+ runs with external content	NEW — core security upgrade
Capability-based access	Each tool invocation requires a capability token scoped to current user request. Token generated at run creation, not inherited from external content.	Every tool call	NEW — replaces implicit trust
Output validation	Separate LLM call (different model, different context) validates that outbound actions are consistent with user's original request and not injection result	Every outbound side-effect	NEW — catch injection at output
Output validation (Pydantic)	Schema validation on LLM structured outputs. Retry on failure.	Every LLM call	Unchanged
Content moderation	OpenAI Moderation API on all inputs and outputs	Every message	Unchanged
Token encryption	AES-256-GCM. Key in AWS Secrets Manager. Never logged or passed to LLM.	Token storage/retrieval	Unchanged
Idempotency	SHA256(run_id + step_id + tool + args). DB unique constraint.	Every tool call	Unchanged
Row-Level Security	Postgres RLS on ALL 19 tables. app.user_id session variable.	Every DB query	Extended to 4 new tables
Rate limiting	Per-user 60/min, per-IP 100/min. Gradual degradation.	Every request	Unchanged
Webhook validation	HMAC-SHA256 (WhatsApp) + cert chain (Apple) + Slack signature	Every webhook	Unchanged
PII redaction	Redact emails, phones, names from logs. Keep in DB only.	All logging	Unchanged
Knowledge file isolation	RLS + user_id scoping. No cross-user file access.	Every knowledge file op	Unchanged
Profiling data protection	Extracted facts stored only in user-scoped tables. Never in shared caches.	Profiling pipeline	Unchanged
Prompt injection fuzzing	Automated adversarial testing in CI/CD pipeline. Tests injection via email, calendar, web content.	Every PR + nightly	NEW — continuous, not Month 9 only
MCP server sandboxing	MCP servers run in isolated containers with scoped network access and pre-approved tool lists	Every MCP call	NEW

32.1 Privilege Isolation Architecture

```
class PrivilegeContext:
    """Defines what tools/actions are available based on content provenance"""

    PRIVILEGE_LEVELS = {
        ContentProvenance.USER_DIRECT: {
            "allowed_tools": "*",           # All tools available
            "can_send_email": True,
            "can_modify_calendar": True,
            "can_access_financial": True,
        },
        ContentProvenance.EMAIL_BODY: {
            "allowed_tools": ["google_calendar.list", "google_contacts.search", "web_search"],
            "can_send_email": False,         # CANNOT send email from email context
            "can_modify_calendar": False,    # CANNOT modify calendar from email context
            "can_access_financial": False,
        },
        ContentProvenance.WEB_SEARCH: {
            "allowed_tools": ["web_search"],  # Can only do more searches
            "can_send_email": False,
            "can_modify_calendar": False,
            "can_access_financial": False,
        },
        ContentProvenance.CALENDAR_DESC: {
            "allowed_tools": ["google_calendar.list"],
            "can_send_email": False,
            "can_modify_calendar": False,
            "can_access_financial": False,
        },
    }

async def execute_with_privilege(tool_call: ToolCall, context: RunContext) -> ToolResult:
    """Enforce privilege isolation before tool execution"""
    privilege = PrivilegeContext.PRIVILEGE_LEVELS[tool_call.input_provenance]

    if privilege["allowed_tools"] != "*" and tool_call.tool_name not in privilege["allowed_tools"]:
        raise PrivilegeViolation(
            f"Tool {tool_call.tool_name} not allowed for content from {tool_call.input_provenance}. "
            f"This may indicate a prompt injection attempt."
        )

    # Validate against capability token
    if not context.capability_token.allows(tool_call.tool_name):
        raise CapabilityViolation(f"Capability token does not grant {tool_call.tool_name}")

    return await tool_executor.execute(tool_call)
```

37. Cost Model & Unit Economics

37.1 Per-Interaction Cost Breakdown (Updated for Multi-Provider)

Component	T0 (15%)	T1 (55%)	T2 (25%)	T3 (5%)	Blended	v3.2 Delta
LLM: Classification	$0	$0.001	$0.001	$0.001	$0.0008	—
LLM: Main inference	$0	$0.003	$0.035	$0.120	$0.0095	-15% via routing
LLM: Embeddings	$0	$0.0002	$0.0005	$0.001	$0.0003	—
LLM: Knowledge file ops	$0	$0.0005	$0.001	$0.002	$0.0006	—
Multi-modal processing	$0	$0.001	$0.002	$0.003	$0.0012	NEW
Tool API calls	$0	$0.001	$0.003	$0.010	$0.0017	—
Compute (amortized)	$0.0005	$0.001	$0.002	$0.005	$0.0013	—
Database queries	$0.0001	$0.0004	$0.0012	$0.003	$0.0005	—
TOTAL	$0.0006	$0.0071	$0.045	$0.145	$0.016	-24% vs v3.2

37.2 Behavioral Intelligence Overhead

Activity	Frequency	Cost Per User/Day	Monthly (1K Users)
Profiling extraction (days 1-30)	~1 session/day for 30 days	$0.008/session	~$240 (one-time ramp)
Nightly consolidation	Daily	$0.015/run	~$450
Weekly self-review	Weekly	$0.02/run	~$80
Correction processing	~2/day average	$0.003/correction	~$180
Research jobs (opt-in)	Per schedule	$0.05-0.50/run	~$300 (varies)
Delegation tracking	Per delegation	$0.005/check	~$50
Knowledge file cache (Redis)	Continuous	Included in Redis cost	$0
TOTAL BEHAVIORAL OVERHEAD			~$130/month + research

Blended cost with behavioral intelligence: $0.019/interaction (was $0.021 in v3.2, down 10% via multi-provider routing). At 50 interactions/user/day and $19.99/month: 94% gross margin at 10K users.

38. Unified Build Plan — Sprint-Level Detail [REVISED]

Each phase delivers production-quality milestones. No throwaway code. The 4-layer Behavioral Intelligence system is built in parallel with the core agent, with integration points at each phase. **v4.0 changes:** Multi-provider LLM router is foundational. Voice processing added. Security privilege isolation added before external content processing. Team/delegation. Workflows and research.

PHASE 1: Foundation

Core Agent

```
M1 S1-2:  AWS VPC, ECS, RDS Postgres + pgvector, Redis. CI/CD. Monitoring (Axiom + OTEL).
M1 S1-2:  Multi-Provider LLM Router v1: unified interface, OpenAI + health monitoring. [NEW - FOUNDATIONAL]
M1 S3-4:  Gateway Plane: WhatsApp webhook, signature validation, dedup, typing indicator, session manager.
M1 S3-4:  Streaming progress reporter (SSE for long-running tasks). [NEW]
M1-2 S5-8: Google Calendar connector: list, create, update, delete, find_free_slots. OAuth. Delta-sync.
M2 S9-10: Brain Plane v1: Tier Router (T0+T1), Intent classifier (GPT-4o-mini), Context compiler (T1 budget).
M2 S9-10: LLM Router v2: Add Anthropic Claude as failover. Task-optimized routing table. [NEW]
M2 S11-12: Google Gmail connector: list, search, get, draft. Delta-sync (5min).
M2 S11-12: Voice-in pipeline: Whisper integration for WhatsApp voice notes. [NEW]
M3 S13-14: Image processing pipeline: GPT-4o Vision for business cards, receipts, screenshots. [NEW]
M3 S15-16: ACE Engine v1: action classification, approval flow (WhatsApp buttons), cold-start Beta priors.
```

Behavioral Intelligence (Parallel)

```
M2 S9-10:  Database schema for ALL 19 tables (including 4 new: delegations, research_jobs,
           workflows, knowledge_graph_edges). Migrations + RLS.
M2 S11-12: Knowledge file templates: USER.md, SOUL.md, IDENTITY.md, AGENTS.md, MEMORY.md,
           HEARTBEAT.md, TOOLS.md, TEAM.md, WORKFLOWS.md. Template engine. [EXPANDED]
M3 S13-14: Profiling Conversation Engine: question generation, fact extraction (GPT-4o-mini),
           knowledge file update pipeline. First 4 dimensions.
M3 S15-16: Context Injection Engine: tier-aware file selection, token budget management.
           Replace static system prompt with dynamic assembly.
M3 S17-18: Onboarding becomes first profiling session. Conversation design with SOUL.md integration.
```

PHASE 2: Intelligence

Core Agent

```
M4 S1-2:  Tier 2 (ReAct loop): multi-step reasoning with adaptive iteration limits. [ENHANCED]
M4 S1-2:  Self-reflection loop after T2+ completion. [NEW]
M4 S3-4:  Email sending: approval flow, recipient verification, draft-review-send cycle.
M4-5 S5-8: Microsoft 365 connector: Calendar + Outlook + Contacts.
M5 S9-10: Proactive Intelligence Engine: morning briefing, pre-meeting prep, follow-up nudges.
M5 S9-10: Privilege isolation for external content (emails, web, calendar descriptions). [NEW - CRITICAL]
M5-6 S11-14: Web search (Tavily). Tier 3 (multi-step planning): Temporal workflow, hierarchical
             decomposition, plan + critic + checkpoints + saga compensation. [ENHANCED]
M6 S15-16: Emotion classifier on inbound messages. Adaptive response tone. [NEW]
M6 S17-18: TEAM.md population from contacts + profiling. Basic delegation engine. [NEW]
```

Behavioral Intelligence (Parallel)

```
M4 S1-2:  Remaining Brain dimensions: people, resources, friction, communication, cognition, content.
M4 S3-4:  DNA layer: AGENTS.md generation from profiling (automation, risk_tolerance, boundaries,
          delegation_preferences). Behavioral rules population. [ENHANCED]
M5 S5-6:  Feedback integration pipeline: correction processing, lesson extraction,
          SOUL.md/AGENTS.md auto-update.
M5 S7-8:  Behavioral inference engine: implicit signal detection from run outcomes, approval patterns,
          edit patterns.
```

M6 S11-12: ACE Engine upgrade: AGENTS.md rules override Beta distributions. Dual Memory Engine path.
M6 S13-14: Memory improvements: knowledge file routing, implicit preference learning,
contact graph enrichment. Knowledge graph edge population. [ENHANCED]
M6 S15-18: Agentic eval suite: 50 golden scenarios + personalization quality scoring.
Prompt injection fuzzing in CI/CD. [ENHANCED]

PHASE 3: Expansion

Core Agent

M7 S1-4: Apple Messages for Business: webhook, iMessage-native UX.
M7 S5-6: Slack connector: read/send, channel summaries.
M7 S5-6: User-defined workflow engine: natural language → trigger/action chain. WORKFLOWS.md. [NEW]
M7-8 S7-10: Advanced scheduling: multi-person availability, timezone intelligence, conflict resolution.
Uses TEAM.md for stakeholder preferences. [ENHANCED]
M8 S11-14: MCP client hub: server discovery, registration, sandboxed execution. [NEW]
M8 S11-14: Semantic caching. Prompt caching. Model cascade optimization.
M9 S15-16: Financial connector (Plaid). High-risk approval flow.
M9 S15-16: Autonomous research engine: Temporal workflows, persistent findings, scheduled delivery. [NEW]
M9 S17-18: Red-team eval: prompt injection via email/calendar/web, wrong recipient, data exfil,
privilege escalation, MCP server attacks. [ENHANCED]

Behavioral Intelligence (Parallel)

M7 S1-2: Nightly consolidation job (BullMQ). Pattern extraction + knowledge compounding.
Contradiction detection across knowledge files. [ENHANCED]
M7 S3-4: Weekly self-review (BullMQ). Gap detection, question generation, completeness tracking.
M8 S5-8: HEARTBEAT system: goal tracking, milestone management, background actions, delegation tracking.
HEARTBEAT.md feeds Proactive Trigger Engine. [ENHANCED]
M8 S9-10: MEMORY.md rules: retention logic, pruning, knowledge compounding.
M9 S11-14: Bones layer: repository scanning, SKILL.md generation, TOOLS.md mapping, MCP catalog. [ENHANCED]
M9 S15-16: Muscles layer: model inventory, cost routing, budget tracking, circuit breakers,
multi-provider health monitoring. [ENHANCED]

PHASE 4: Polish & Scale

Core Agent

M10 S1-4: Document generation engine: Markdown → PDF/DOCX, template-based output. [NEW]
M10 S1-4: Voice TTS output: audio briefings, hands-free mode. [NEW]
M10 S1-4: Travel connectors, smart home, Notion/Google Docs search.
M10-11 S5-8: Advanced proactive intelligence with HEARTBEAT-driven triggers + research delivery.
M10-11 S5-8: Cross-channel context continuity: unified session, channel preference learning. [NEW]
M11 S9-12: Performance: p95 < 3s, cost < $0.035/interaction, cache > 20%.
LLM Router optimization: latency-based routing, cost-based routing for batch. [ENHANCED]
M11-12 S13-14: Load test at 10K concurrent. Graceful degradation. Multi-provider failover under load.
M12 S15-16: Security hardening: pentest, OWASP LLM Top 10, privilege isolation audit.
M12 S17-18: Production launch: runbooks, incident playbooks, on-call, documentation.

Behavioral Intelligence (Parallel)

M10 S1-2: 'What do you know about me?' command. Knowledge file viewer/editor via chat.
Knowledge graph query interface. [ENHANCED]
M10 S3-4: Redis caching optimization. Pre-computed context blocks. Batch file updates.
M11 S5-8: Personalization eval: profiling coverage, knowledge accuracy, correction frequency,
user satisfaction, delegation success rate, research quality. [ENHANCED]
M11-12 S9-12: Cross-user anonymized insights (opt-in): popular tools, common friction patterns,
model cost optimization, delegation patterns.
M12 S13-16: A/B testing framework: experiment tracking, variant assignment, outcome metrics. [NEW]
M12 S13-16: Documentation: knowledge file format spec, profiling question bank (150+), signal catalog,
MCP integration guide, delegation protocol, research API guide. [ENHANCED]
M12 S17-18: Data export: GDPR/CCPA compliant user data export endpoint. [NEW]

33. Observability & Monitoring

Metric	Target	Alert Threshold	Source	v4.0 Notes
p95 latency (WhatsApp e2e)	< 3s	> 5s for 5min	OTEL traces	Includes multi-modal preprocessing
Error rate (5xx)	< 0.1%	> 1% for 5min	HTTP status codes	Per-provider error tracking
LLM call success rate	> 99.5%	< 98% for 10min	LLM Router logs	Per-provider + aggregate
LLM failover rate	< 2%	> 10% for 30min	LLM Router logs	NEW — tracks provider health
Tool execution success	> 98%	< 95% for 10min	tool_executions table	Includes MCP tools
Cost per interaction (blended)	< $0.035	> $0.05 for 1hr	Cost tracking	Multi-provider cost rollup
Tier distribution	T0:15% T1:55% T2:25% T3:5%	T3 > 10% for 1hr	Tier Router	—
Semantic cache hit rate	> 20%	< 10% for 1day	Cache metrics	—
Knowledge file completeness (avg)	> 60% by day 30	< 30% after day 14	knowledge_files table	Includes TEAM.md, WORKFLOWS.md
Profiling completion rate	> 80% of scheduled	< 50% for 1 week	profiling_sessions	—
Correction frequency	Declining trend	Increase > 20% wow	feedback_signals	—
Proactive msg helpful rate	> 50%	< 30% for 1 week	Feedback signals	—
Voice transcription accuracy	> 95%	< 90% for 1day	Whisper confidence	NEW
Delegation on-time rate	> 80%	< 60% for 2 weeks	delegations table	NEW
Research job success rate	> 90%	< 70% for 1 week	research_jobs table	NEW
Prompt injection blocks	Informational	> 5/hour	Security logs	NEW
DAU / MAU ratio	> 50%	< 30% for 2 weeks	Analytics	—

34. Testing Strategy — Unit to Red-Team

Test Type	What It Validates	Framework	Frequency	Blocking ?	v4.0 Notes
Unit tests	Functions: context compiler, memory retrieval, tier router, autonomy scoring, knowledge file merge, profiling extraction, LLM router selection	pytest + hypothesis	Every PR	Yes	Added LLM router, privilege isolation
Integration tests	Service-to-service: Gateway→Brain, Brain→Hands, knowledge pipeline, dual memory path, multi-modal pipeline	pytest + testcontainers	Every PR	Yes	Added multi-modal, MCP
Contract tests	API contracts: Pydantic schemas, OpenAPI spec, tool schemas, knowledge file schemas, MCP server schemas	schemathesis	Schema-change PRs	Yes	Added MCP contracts
Connector tests	Each tool: golden I/O pairs, record/replay, error handling, MCP server responses	pytest + VCR.py	Every PR + nightly	Yes	Added MCP server tests
Agentic eval	50 → 100 golden scenarios covering all tiers, intents, knowledge file injection quality, multi-modal inputs	Braintrust eval	Nightly + pre-deploy	Yes (score drop)	Expanded to 100 scenarios
Personalization eval	Knowledge file accuracy, profiling extraction quality, correction incorporation speed, delegation tracking accuracy	Custom eval harness	Weekly	No (informational)	Added delegation, team evals
Red-team eval	Prompt injection via email/calendar/web/MCP, jailbreak, data exfil, hallucination, cross-user data leak, privilege escalation	Custom + HarmBench	Weekly + CI/CD	Yes (vulnerability)	ENHANCED: continuous fuzzing
Load test	100, 1K, 10K concurrent. Latency under load. Knowledge file serving under load. LLM failover under load	k6 + Grafana	Phase milestones	No	Added failover load testing
Chaos test	Kill Brain, kill Redis, throttle Postgres, LLM timeout, knowledge file cache failover, provider outage simulation	Custom fault injection	Monthly	No	Added provider outage sim

A. Appendix A: Environment Variables & Config

```
# === Gateway ===
WA_VERIFY_TOKEN=                    # WhatsApp webhook verification token
WA_APP_SECRET=                      # WhatsApp app secret (HMAC validation)
WA_ACCESS_TOKEN=                    # WhatsApp Cloud API access token
WA_PHONE_NUMBER_ID=                 # WhatsApp business phone number ID
WA_BUSINESS_ACCOUNT_ID=             # WhatsApp business account ID
APPLE_BUSINESS_CHAT_ID=             # Apple Messages for Business ID
SLACK_BOT_TOKEN=                    # Slack bot token
SLACK_SIGNING_SECRET=               # Slack request signing secret
CLERK_SECRET_KEY=                   # Clerk auth secret

# === Brain (Multi-Provider) ===
OPENAI_API_KEY=                     # OpenAI API key (GPT-4o + GPT-4o-mini + embeddings + Whisper + TTS)
OPENAI_ORG_ID=                      # OpenAI organization ID
ANTHROPIC_API_KEY=                  # Anthropic API key (Claude Sonnet)           [NEW]
GOOGLE_AI_API_KEY=                  # Google AI API key (Gemini Flash/Pro)        [NEW]
LOCAL_LLM_ENDPOINT=                 # Local vLLM endpoint (Llama 4 / Mistral)    [NEW]
LLM_ROUTER_FAILOVER_TIMEOUT_S=30    # Seconds before marking provider unhealthy  [NEW]
LLM_ROUTER_HEALTH_CHECK_INTERVAL=30 # Health check frequency in seconds         [NEW]

# === Hands ===
GOOGLE_CLIENT_ID=                   # Google OAuth client ID
GOOGLE_CLIENT_SECRET=               # Google OAuth client secret
MICROSOFT_CLIENT_ID=                # Microsoft OAuth client ID
MICROSOFT_CLIENT_SECRET=            # Microsoft OAuth client secret
TAVILY_API_KEY=                     # Tavily web search API key
ELEVENLABS_API_KEY=                 # ElevenLabs TTS API key (optional)         [NEW]
PLAID_CLIENT_ID=                    # Plaid financial connector
PLAID_SECRET=                       # Plaid secret

# === Data ===
DATABASE_URL=                       # postgresql://user:pass@host:5432/executive_os
REDIS_URL=                          # redis://host:6379/0

# === Infrastructure ===
AWS_REGION=us-east-1
AWS_SECRETS_PREFIX=executive-os/prod/
S3_BUCKET=executive-os-attachments
S3_KNOWLEDGE_BUCKET=executive-os-knowledge-snapshots
S3_VOICE_BUCKET=executive-os-voice                                              [NEW]
S3_DOCUMENTS_BUCKET=executive-os-documents                                      [NEW]
AXIOM_API_TOKEN=
AXIOM_DATASET=executive-os-prod

# === Feature Flags ===
FEATURE_TIER_3_ENABLED=true
FEATURE_PROACTIVE_ENABLED=true
FEATURE_IMESSAGE_ENABLED=false
FEATURE_BEHAVIORAL_INTELLIGENCE=true
FEATURE_PROFILING_ENABLED=true
FEATURE_CONSOLIDATION_ENABLED=true
FEATURE_SELF_REVIEW_ENABLED=true
FEATURE_MULTI_PROVIDER_LLM=true                  # NEW: enable multi-provider routing
FEATURE_VOICE_INPUT=true                          # NEW: enable voice note processing
FEATURE_VOICE_OUTPUT=false                        # NEW: enable TTS responses
FEATURE_IMAGE_PROCESSING=true                     # NEW: enable image understanding
FEATURE_DOCUMENT_PROCESSING=true                  # NEW: enable document ingestion
FEATURE_TEAM_DELEGATION=false                     # NEW: enable team features
FEATURE_RESEARCH_ENGINE=false                     # NEW: enable autonomous research
FEATURE_WORKFLOWS=false                           # NEW: enable user-defined workflows
FEATURE_MCP_CLIENT=false                          # NEW: enable MCP integration
FEATURE_EMOTION_DETECTION=true                    # NEW: enable emotion-aware responses
FEATURE_PRIVILEGE_ISOLATION=true                  # NEW: enable content provenance security
FEATURE_DOCUMENT_GENERATION=false                 # NEW: enable doc generation output
```

```
FEATURE_CROSS_CHANNEL_CONTINUITY=false            # NEW: enable unified sessions
FEATURE_AB_TESTING=false                          # NEW: enable experiment framework
```

H. Appendix H: Multi-Provider Failover Matrix [NEW]

This matrix defines the failover behavior for every task type when the primary LLM provider is unavailable. The LLM Router uses this matrix in combination with real-time provider health checks.

Task Type	Primary	Failover 1	Failover 2	Degraded Mode
Intent classification	GPT-4o-mini	Gemini Flash	Llama 4 (local)	Regex patterns (T0 only)
T1 single tool call	GPT-4o-mini	Claude Sonnet	Gemini Flash	Queue + retry
T2 ReAct reasoning	GPT-4o	Claude Sonnet	Gemini Pro	Degrade to T1
T3 multi-step plan	GPT-4o	Claude Sonnet	—	Queue + notify user
Email drafting	Claude Sonnet	GPT-4o	Gemini Pro	Template-based fallback
Knowledge extraction	GPT-4o-mini	Gemini Flash	Llama 4 (local)	Skip extraction, log
Knowledge compilation	GPT-4o-mini	Gemini Flash	—	Defer to next consolidation
Profiling extraction	GPT-4o-mini	Claude Sonnet	—	Pause profiling session
Safety validation	Claude Sonnet	GPT-4o	—	Block action, require user confirm
Voice transcription	Whisper (OpenAI)	Gemini Flash (audio)	—	Queue audio, retry later
Image understanding	GPT-4o Vision	Gemini Pro Vision	—	Store image, process later
Embeddings	text-embedding-3-small	Gemini embedding	—	Skip embedding, text search only
PII processing	Llama 4 (local)	GPT-4o (with redaction)	—	Block, require local model

— END OF BLUEPRINT —

This document is the single source of truth for the Executive OS v4.0 implementation. All code, schemas, API specifications, and build timelines are production-ready. The plan delivers incrementally — every sprint produces deployable, tested artifacts. No throwaway code. No prototypes-that-become-production. The behavioral intelligence system, multi-modal pipeline, multi-provider LLM router, team delegation engine, and security privilege isolation collectively make this a true digital chief of staff, not just a chatbot with calendar access.